PHASE 3 REPORT

Big Data Applications

PRESENTED BY

Đặng Văn Kỳ

Nguyễn Văn Minh Thiện

INSTRUCTOR

Cô Nguyễn Ngọc Thảo

Thây Nguyễn Thanh Tình

Thây Huỳnh Lâm Hải Đăng

CONTENTS

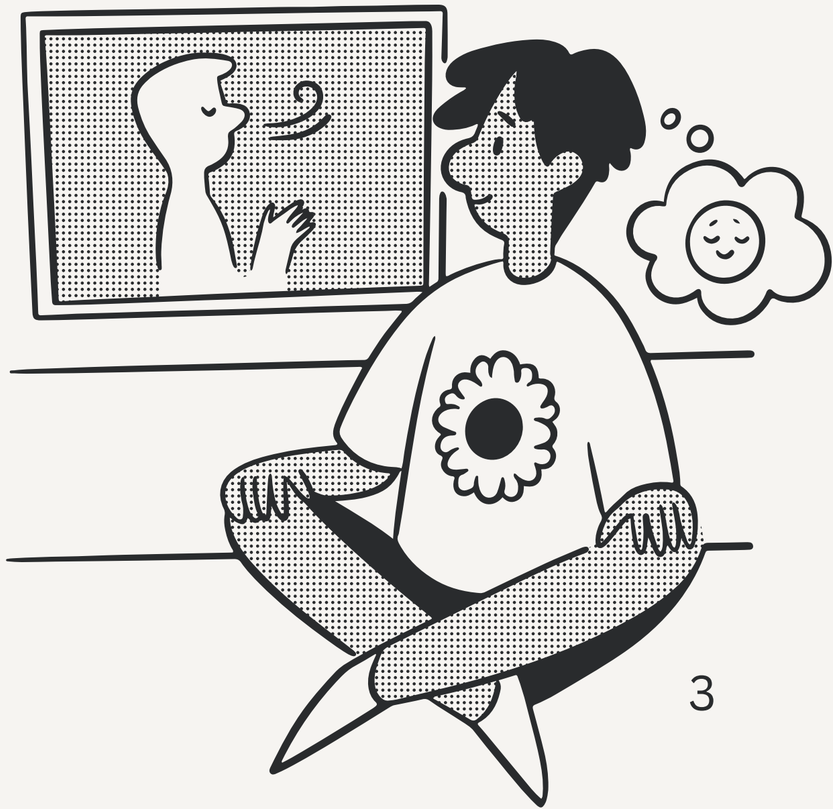
- Introduction & Problem Statement
- LightGCN
- SGL
- SimGCL
- Datasets
- SOTA Models Comparison
- Datasets
- Data Pipeline
- Experiments
- Results
- Analysis & Insight
- Conclusion

INTRODUCTION & PROBLEM STATEMENT

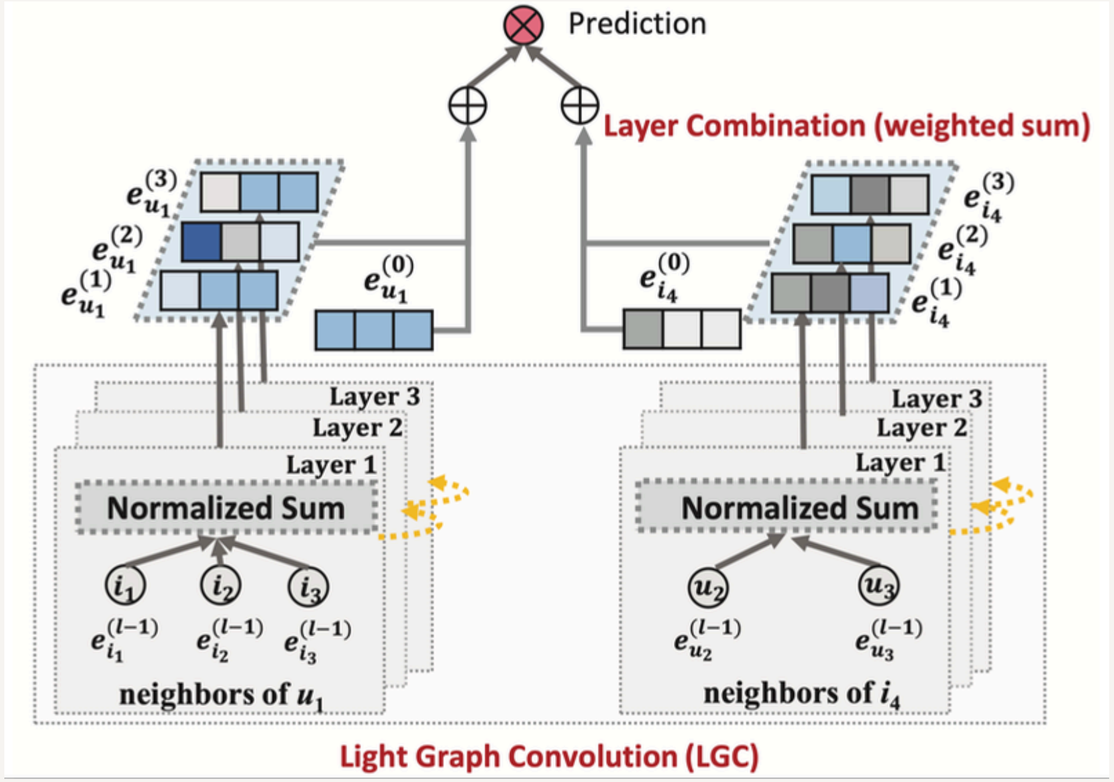
RECOMMENDATION SYSTEM WITH IMPLICIT FEEDBACK

- Modern platforms mostly collect implicit feedback (click, view, purchase, watch time), not explicit ratings.
- We study recommendation in the implicit-feedback setting with binary interactions.
- Task: Top-K recommendation for each user.

.

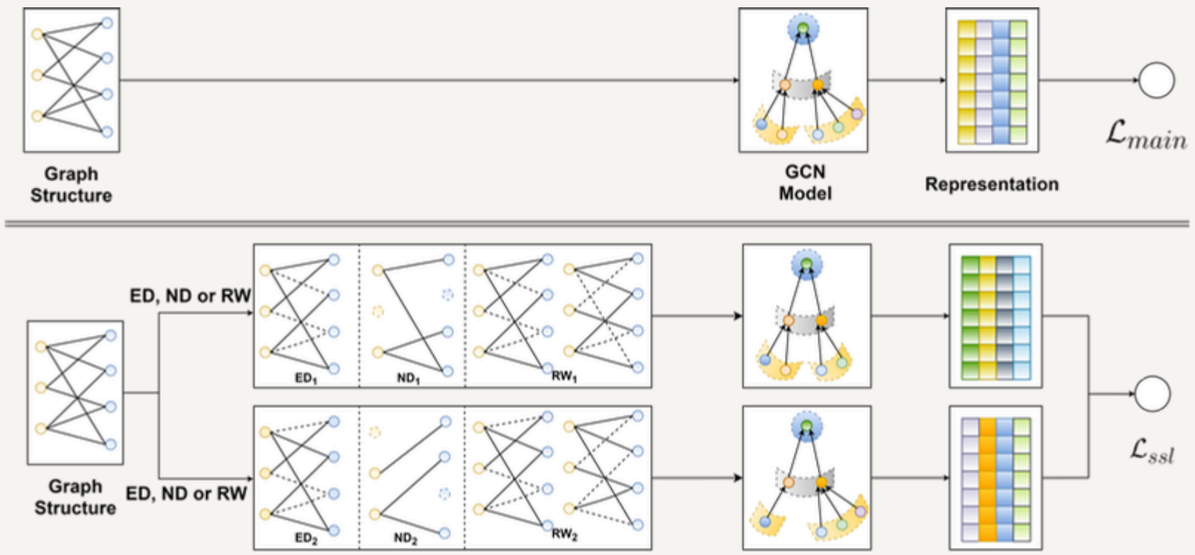


LightGCN



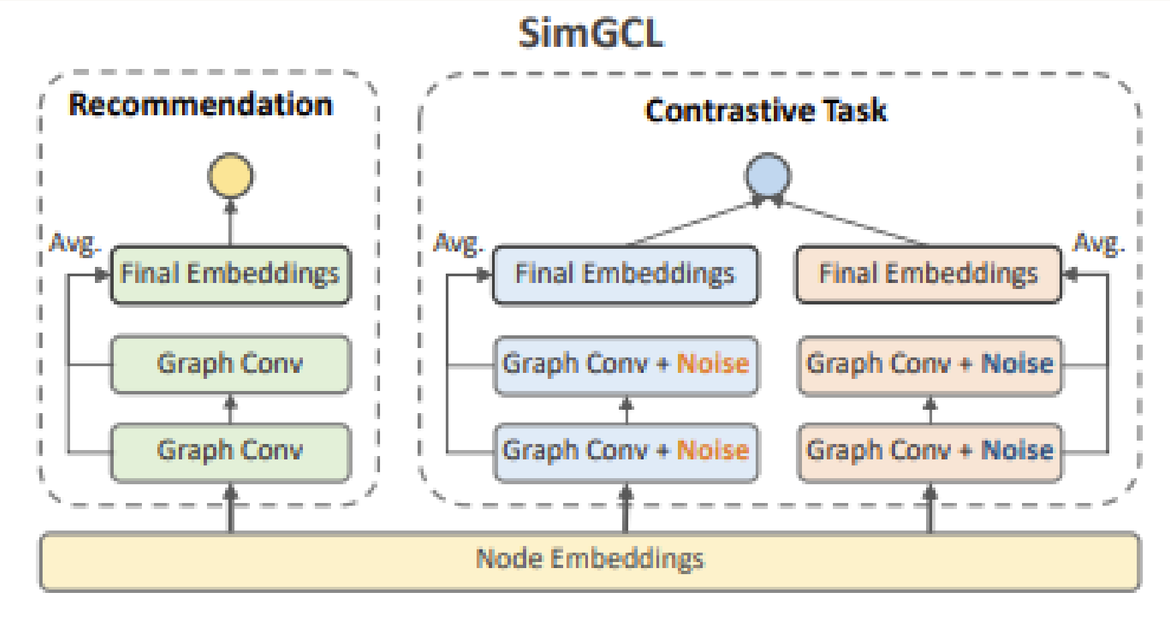
- Core idea: Simplify GCN by removing feature transformation and nonlinearity.
- Architecture: neighbor aggregation on user-item bipartite graph + layer combination.
- Training objective: BPR loss.
- Reasons: Strong, lightweight, highly reproducible baseline.
- https://github.com/gusye1234/LightGCN-PyTorch

SGL



- Core idea: Add self-supervised contrastive learning to graph recommendation.
- Architecture: Graph augmentations (node/edge dropout or random walk) + InfoNCE + BPR.
- Innovation: Improves robustness under sparsity and long-tail interactions.
- Reasons: Representative SSL-based breakthrough over LightGCN.
- https://github.com/wujcan/SGL-Torch

SimGCL



- Core idea: Graph augmentation is not necessary; use embedding noise perturbation.
- Architecture: LightGCN backbone + two noisy views + InfoNCE + BPR.
- Innovation: Simpler and often stronger than augmentation-heavy SSL methods.
- Reasons: Strong accuracy-efficiency trade-off with simple implementation.
- https://github.com/wujcan/SGL-Torch

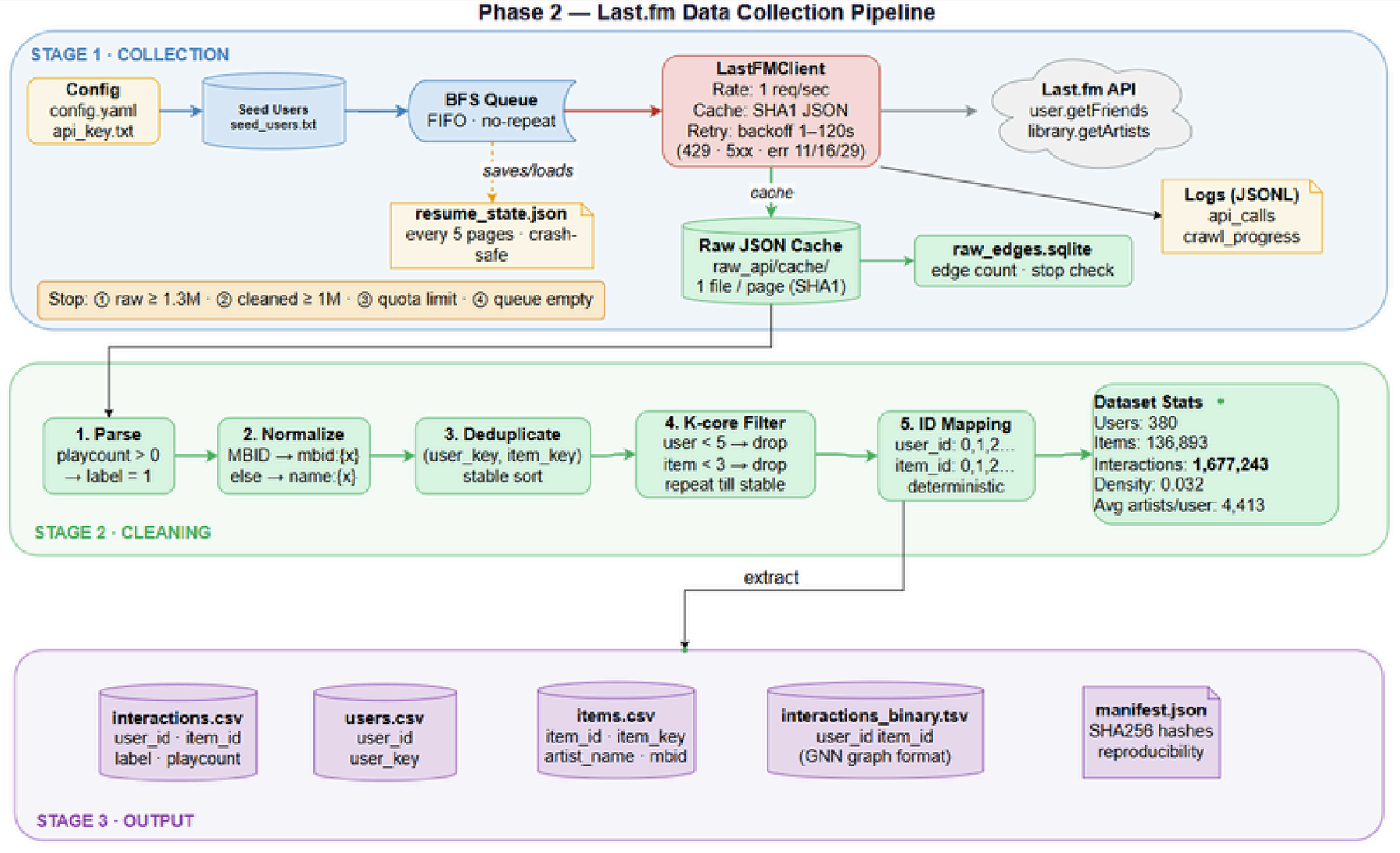
SOTA MODELS Comparison

LightGCN	SGL	SimGCL
• Simplest and fastest baseline. • No SSL auxiliary signal.	• Strongest robustness on sparse data. • Higher training complexity.	• Near-LightGCN efficiency with SSL gains and better scalability.

DATASETS

- Source: Last.fm Web API.
- **user.getFriends** is endpoint used to expand the user graph with BFS.
- **library.getArtists** is endpoint is used to collect each user's artist library page by page.
- This source is public, reproducible, and naturally aligned with implicit feedback.
- Main constraints: API rate limits, quota control, and incomplete artist identifiers.

PIPELINE

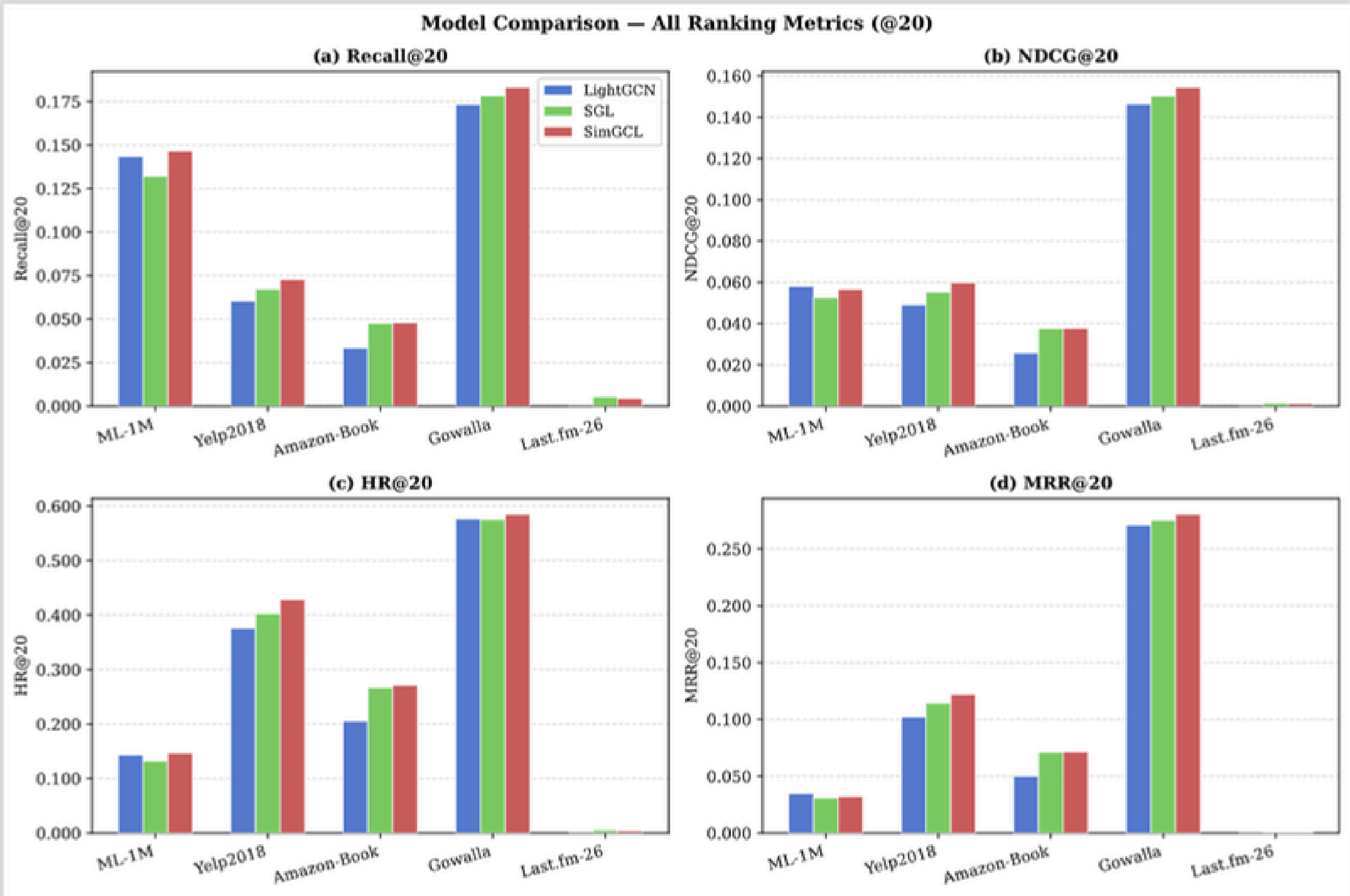

Phase 2 — Last.fm Data Collection Pipeline
STAGE 1 · COLLECTION
Config
config.yaml
api_key.txt
Seed Users
seed_users.txt
BFS Queue
FIFO · no-repeat
saves/loads
resume_state.json
every 5 pages · crash-safe
LastFMClient
Rate: 1 req/sec
Cache: SHA1 JSON
Retry: backoff 1–120s
(429 · 5xx · err 11/16/29)
Last.fm API
user.getFriends
library.getArtists
cache
Raw JSON Cache
raw_api/cache/
1 file / page (SHA1)
raw_edges.sqlite
edge count · stop check
Logs (JSONL)
api_calls
crawl_progress
Stop: ① raw ≥ 1.3M · ② cleaned ≥ 1M · ③ quota limit · ④ queue empty
1. Parse
playcount > 0
→ label = 1
2. Normalize
MBID → mbid:{x}
else → name:{x}
3. Deduplicate
(user_key, item_key)
stable sort
4. K-core Filter
user < 5 → drop
item < 3 → drop
repeat till stable
5. ID Mapping
user_id: 0,1,2…
item_id: 0,1,2…
deterministic
Dataset Stats
Users: 380
Items: 136,893
Interactions: 1,677,243
Density: 0.032
Avg artists/user: 4,413
STAGE 2 · CLEANING
extract
interactions.csv
user_id · item_id
label · playcount
users.csv
user_id
user_key
items.csv
item_id · item_key
artist_name · mbid
interactions_binary.tsv
user_id item_id
(GNN graph format)
manifest.json
SHA256 hashes
reproducibility
STAGE 3 · OUTPUT

EXPERIMENT

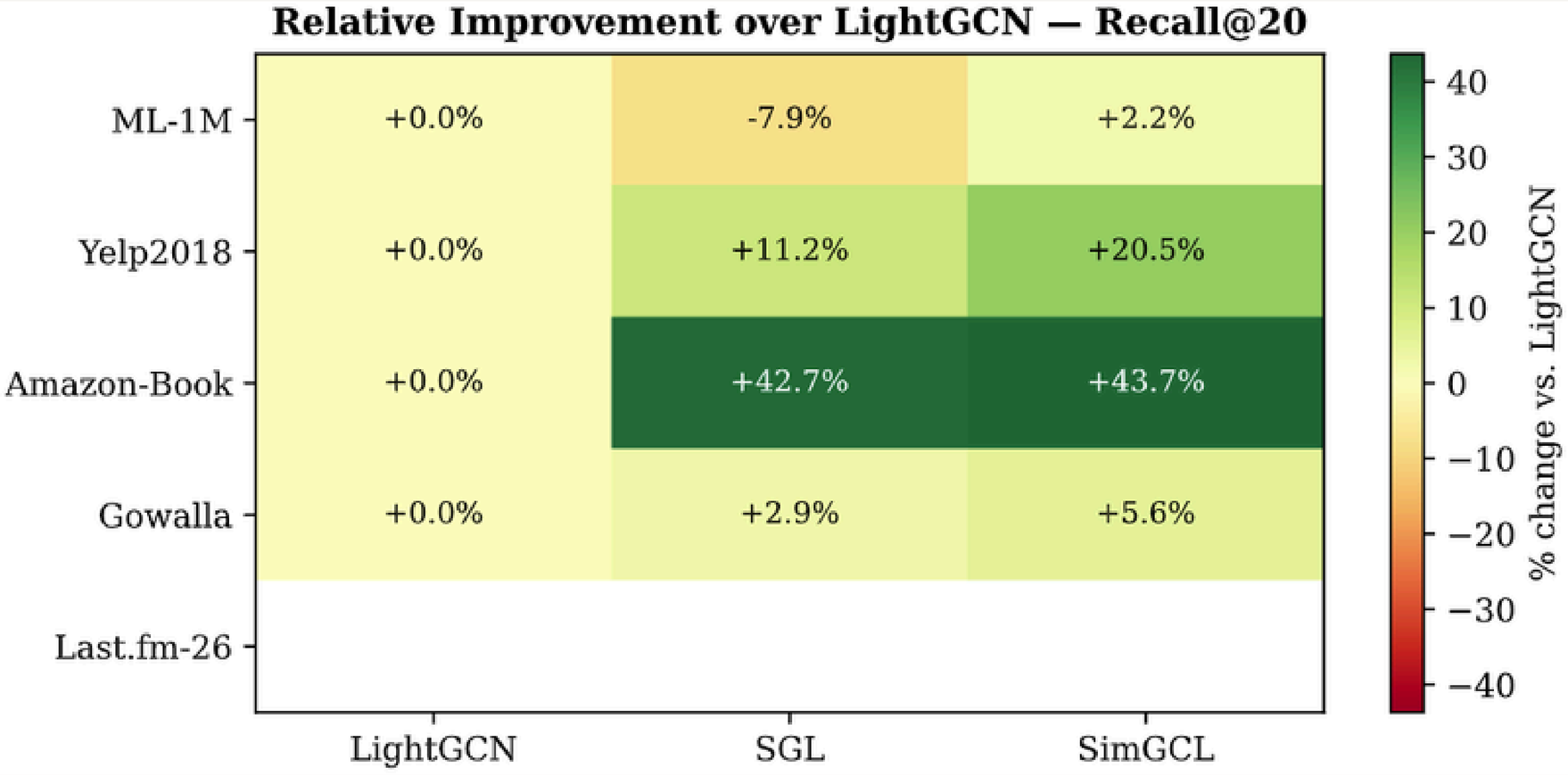
Dataset
Last.fm-26 (Ours)
MovieLens 1M
Amazon-book
Yelp2018
Gowalla

- Run 3 SOTA Models on 5 datasets.
- Each dataset run 3 times and compute the average results.
- Use 4 main metrics to evaluation: Recall@20, HR@20, NDCG@20, MRR@20.

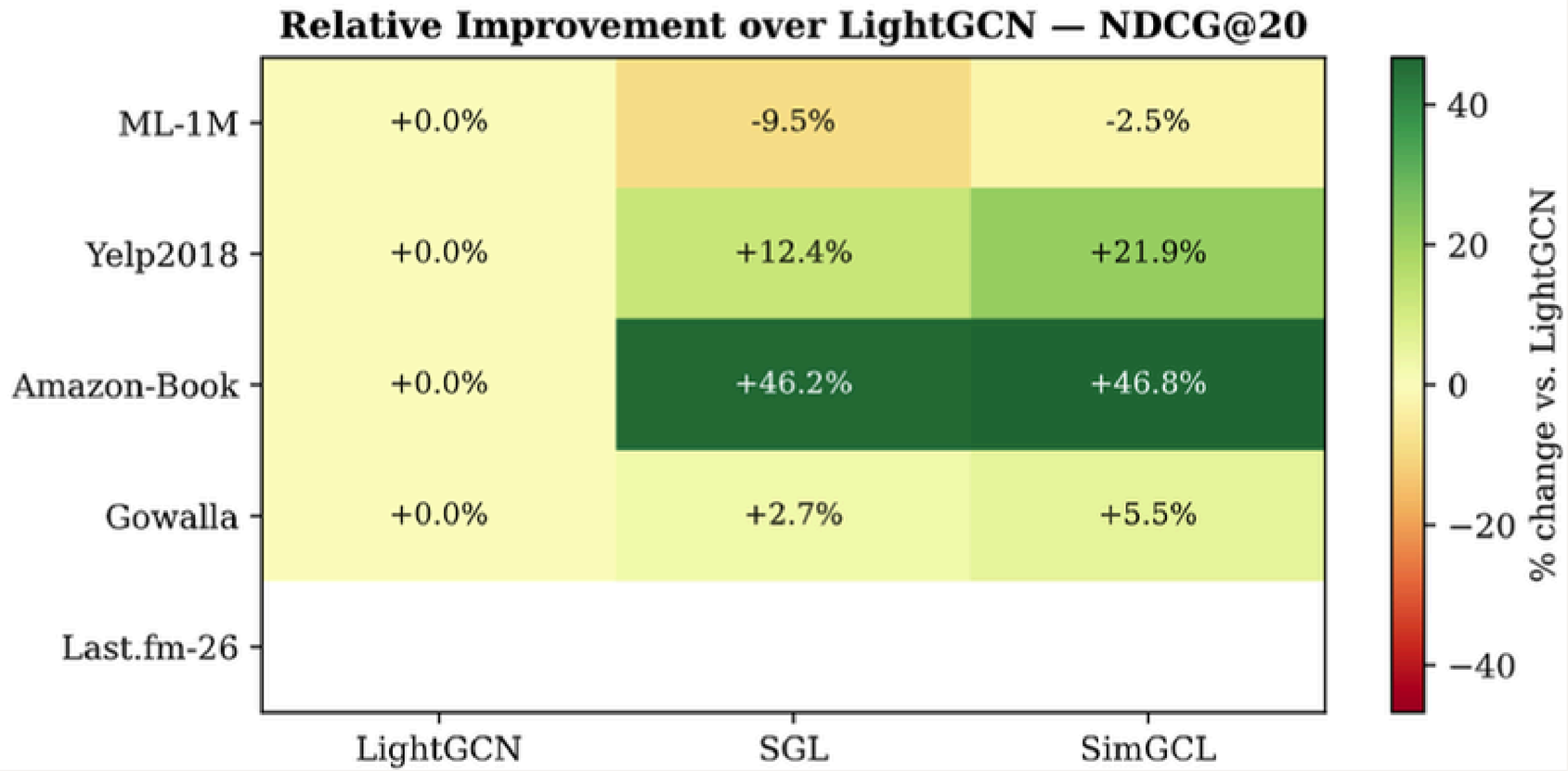
RESULTS


Model Comparison — All Ranking Metrics (@20)
(a) Recall@20
(b) NDCG@20
(c) HR@20
(d) MRR@20
LightGCN
SGL
SimGCL
Recall@20
NDCG@20
HR@20
MRR@20
ML-1M
Yelp2018
Amazon-Book
Gowalla
Last.fm-26

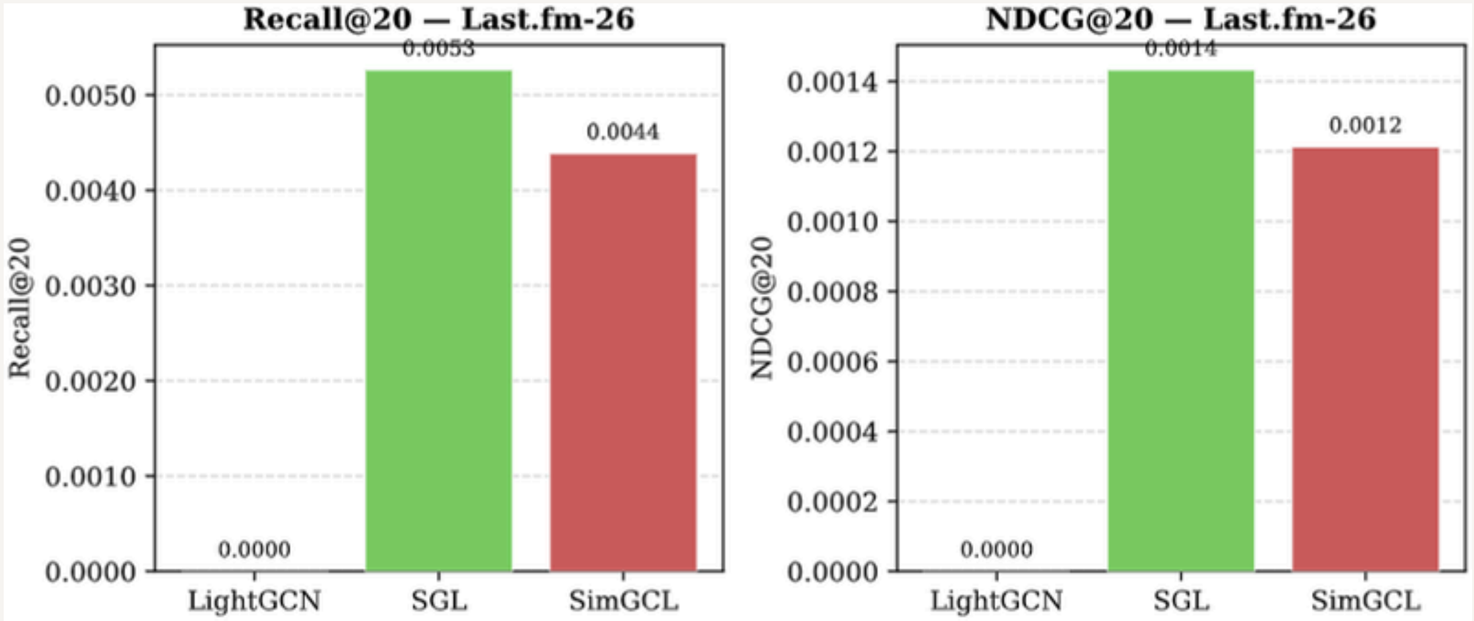
ANALYSIS & INSIGHT



ANALYSIS & INSIGHT

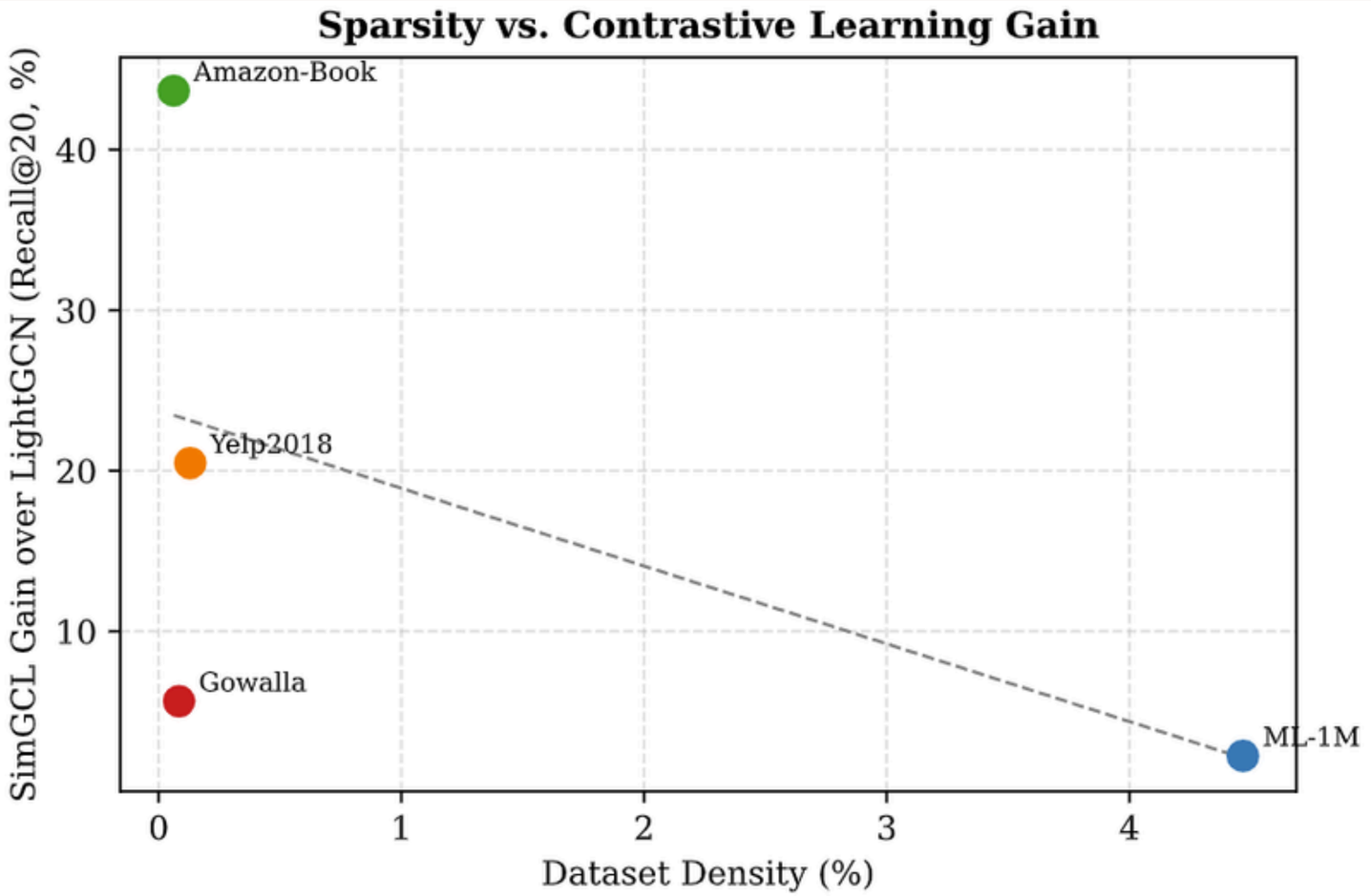


ANALYSIS & INSIGHT



- Last.fm-26 collapse
- 380 users / 136,893 items.
- LightGCN = 0 on all metrics.
- SGL is the best model for sparse dataset.

ANALYSIS & INSIGHT



ANALYSIS & INSIGHT

DATA IMPACT OF LAST.FM-26

- **Item Cold Start**: Most items have only 3 interactions from 380 users — insufficient for GNN embedding.
- **Graph Disconnection:** Items rarely share common users → multi-hop propagation carries near-zero signal
- **K-core Paradox:** Filtering keeps 136,893 items from 380 users → structurally valid but semantically empty graph
- **New User Cold Start:** Only 380 training users → zero generalization capacity for unseen users

CONCLUSION

- **SimGCL** is the most consistently strong model. It achieves the best performance on four out of five datasets and matches **LightGCN's** training efficiency, making it the recommended choice for sparse implicit-feedback collaborative filtering.
- Self-supervised contrastive learning is beneficial on sparse data but harmful on dense data. SGL degrades below **LightGCN** on ML-1M (the densest dataset at 4.47%) due to graph augmentation corrupting rich neighborhood structure. SimGCL mitigates this by operating entirely in embedding space.
- Dataset characteristics — particularly interaction density and user-to-item ratio — are the dominant factor in determining model performance, more so than model architecture in the medium-to-low sparsity regime.

FUTURE WORK

- **SimGCL — Adaptive noise:** replace fixed noise with interaction-density-aware perturbation per node
- **SGL — Learned augmentation:** learn to select edges to drop based on interaction weight instead of random dropout